



UFC

UNIVERSIDADE FEDERAL DO CEARÁ

LUCAS WARLEY RODRIGUES VIANA - 498884

TRABALHO COMPILADORES-(Relatório individual)

RUSSAS

2025

1. Introdução

Este relatório descreve os avanços realizados no projeto de desenvolvimento de um compilador, com foco na construção de uma linguagem de programação personalizada. A tarefa envolve a análise e implementação incremental de componentes essenciais, como a gramática da linguagem, a definição de tipos de dados, e a modularização do código. Na segunda parte do projeto (AV2-P2), o objetivo é expandir a linguagem, incorporando novos tipos de dados, como Listas e Tuplas, além de adicionar outras funcionalidades conforme necessário. Durante este processo, também são realizadas modificações na gramática e no lexer, garantindo que todos os novos elementos sejam reconhecidos corretamente.

A etapa deste relatório enfatiza a análise teórica das possíveis expansões da linguagem, com foco na definição e implementação de novos tipos de dados e funcionalidades. O objetivo é explorar a flexibilidade e a criatividade na evolução do projeto, propondo novas soluções que atendam às necessidades da linguagem, enquanto mantém a integridade e a modularidade do compilador. Além disso, é importante refletir sobre a viabilidade de executar essas inovações no código, sem deixar de lado o impacto teórico das mudanças na gramática e nas funcionalidades da linguagem.

2. Objetivo

O objetivo deste projeto é desenvolver e expandir um compilador para uma linguagem de programação personalizada. Na segunda parte do projeto, o foco está na implementação de novos tipos de dados e funcionalidades, como Listas e Tuplas, além da análise e modificação da gramática da linguagem. O projeto busca aprimorar a compreensão dos conceitos de compilação, desde a criação de tipos de dados até a definição de novas funcionalidades, garantindo a modularidade e flexibilidade da linguagem. Além disso, o trabalho visa refletir sobre as possibilidades de evolução do compilador, com base nas necessidades e inovações teóricas para a linguagem.

3 Desenvolvimento

Problema 1: Implementação do tipo tupla

Lexer.py

```
elif self.current == Consts.LPAR:
    tokens.append(Token(Consts.LPAR))
    self.__advance()
elif self.current == Consts.RPAR:
    tokens.append(Token(Consts.RPAR))
    self.__advance()
```

Grammar.py

```
elif tok.type == Consts.LPAR:
    tuplaExp = ast.registry(TuplaExp(self.parser).Rule())
    if (ast.error!=None): return ast
    return ast.success(tuplaExp)
```

TValue.py

```
class TTuple(TValue):
    def __init__(self, value):
        self.value = tuple(value) # Imutável
        self.setMemory()

    def setMemory(self, memory=None):
        self.memory = memory
        return self

    def add(self, other):
        if isinstance(other, TTuple):
            return TTuple(self.value + other.value).setMemory(self.memory), None
        return super().add(other)

    def getItem(self, other):
        if(isinstance(other,TNumber)):
            return self.value[other.value]

    def copy(self):
        return TTuple(self.value).setMemory(self.memory)

    def __repr__(self):
        return f"({'', '.join(map(str, self.value))})"
```

SematicVisitor.py

```
class NoTupla(Visitor):
    def __init__(self, elements):
        self.elements = elements

    def visit(self, operator):
        tupleValues = [operator.registry(element_node.visit(operator)) for element_node in self.elements]
        return operator.success(TTuple(tupleValues).setMemory(operator))

    def __repr__(self):
        return f"({'', ".join(map(str, self.elements))})"
```

Problema 2: Implementação de novas estruturas de dados como também novas funcionalidades

- Adição dicionário

Lexer.py

```
elif self.current == Consts.LBRACE:
    tokens.append(Token(Consts.LBRACE))
    self.__advance()
elif self.current == Consts.RBRACE:
    tokens.append(Token(Consts.RBRACE))
    self.__advance()
elif self.current == Consts.COLON:
    tokens.append(Token(Consts.COLON))
    self.__advance()
```

Grammar.py

```
elif tok.type == Consts.LBRACE:
    dictExp = ast.registry(DictExp(self.parser).Rule())
    if (ast.error!=None): return ast
    return ast.success(dictExp)
```

```
class DictExp(Grammar):
    def Rule(self):
        ast = self.GetParserManager()
        self.NextToken()
        key_value_pairs = []

        if self.CurrentToken().type != Consts.RBRACE: # Se não for um dicionário vazio '{}'
            key = ast.registry(Exp(self.parser).Rule()) # Obtém a chave
            if ast.error: return ast

            if self.CurrentToken().type != Consts.COLON:
                return ast.fail(f"{Error.parserError}: Esperado ':' após chave do dicionário")

            self.NextToken() # Consome ':'
            value = ast.registry(Exp(self.parser).Rule()) # Obtém o valor
            if ast.error: return ast

            key_value_pairs.append((key, value))

        while self.CurrentToken().type == Consts.COMMA: # Processa múltiplos pares chave-valor
            self.NextToken()
            key = ast.registry(Exp(self.parser).Rule())
            if ast.error: return ast

            if self.CurrentToken().type != Consts.COLON:
                return ast.fail(f"{Error.parserError}: Esperado ':' após chave do dicionário")

            self.NextToken()
            value = ast.registry(Exp(self.parser).Rule())
            if ast.error: return ast

            key_value_pairs.append((key, value))

        if self.CurrentToken().type != Consts.RBRACE:
            return ast.fail(f"{Error.parserError}: Esperado '}' para fechar o dicionário")

        self.NextToken() # Consome '}'
        return ast.success(NoDict(key_value_pairs))
```

TValue.py

```
class TDict(TValue):
    def __init__(self, value):
        self.value = value
        self.setMemory()

    def setMemory(self, memory=None):
        self.memory = memory
        return self

    def getItem(self, other):
        return self.value[other]

    def removeItem(self, other):
        self.value.pop(other)
        return self

    def addIndex(self, other, value):
        self.value[other] = value
        return self.value

    def copy(self):
        copy = TDict(self.value)
        copy.setMemory(self.memory)
        return copy

    def __repr__(self):
        return f"{str(self.value)}"
```

SematicVisitor.py

```
class NoDict(Visitor):
    def __init__(self, key_value_pairs):
        self.key_value_pairs = key_value_pairs # Lista de pares (chave, valor)

    def visit(self, operator):
        dict_value = {}
        for key_node, value_node in self.key_value_pairs:
            key = operator.registry(key_node.visit(operator))
            if operator.error: return operator
            value = operator.registry(value_node.visit(operator))
            if operator.error: return operator
            dict_value[key] = value
        return operator.success(TDict(dict_value).setMemory(operator))

    def __repr__(self):
        return f"NoDict({self.key_value_pairs})"
```

No lexer, criamos a identificação dos tokens e a adição dos mesmos para que possam passar pelo parser e serem validados. No arquivo Grammar.py, é feita a identificação do símbolo {, que indica uma possível declaração de um dicionário. Com isso, é feita a chamada para DictExp para validação e atribuição dos valores do dicionário, com o auxílio de NoDict e TDict.

- Chamada de métodos

Grammar.py

```
class MethodListTuple(Grammar):
    def Rule(self):
        ast = self.GetParserManager()
        tok = self.CurrentToken()
        self.NextToken()
        if self.CurrentToken().type == Consts.POINT:
            self.NextToken() # Avança para o nome do método
            if self.CurrentToken().type != Consts.ID:
                return ast.fail(f"{Error.parserError}: Nome do método esperado após '.'")

            method_name = self.CurrentToken().value # Nome do método (ex: "append")
            self.NextToken()

            if self.CurrentToken().type != Consts.LPAR:
                return ast.fail(f"{Error.parserError}: Esperado '(' após nome do método")

            self.NextToken() # Avança para os argumentos do método
            param_nodes = []

            # Processa os parâmetros dentro dos parênteses
            if self.CurrentToken().type != Consts.RPAR:
                param_nodes.append(ast.registry(Exp(self.parser).Rule()))
                if ast.error:
                    return ast

            if self.CurrentToken().type != Consts.RPAR:
                return ast.fail(f"{Error.parserError}: Esperado ')' para fechar a chamada de método")

            self.NextToken()
            return ast.success(NoMethodCall(NoVarAccess(tok), method_name, param_nodes))
```

SematicVisitor.Py

```
class NoMethodCall(Visitor):
    def __init__(self, object_node, method_name, params, value = None):
        self.object_node = object_node
        self.method_name = method_name
        self.params = params
        self.value = value

    def visit(self, operator):
        obj = operator.registry(self.object_node.visit(operator))
        if operator.error: return operator

        if isinstance(obj, TList) or isinstance(obj, TTuple) or isinstance(obj, TDict):
            if self.method_name == "append" and isinstance(obj, TList):
                if len(self.params) != 1:
                    return operator.fail(Error("Metodo 'append' recebe exatamente 1 argumento"))
                value = operator.registry(self.params[0].visit(operator))
                if operator.error: return operator
                obj.appendItem(value)
                return operator.success(obj)

            if self.method_name == "get":
                chave = operator.registry(self.params.visit(operator))
                if operator.error: return operator
                valor = obj.getItem(chave)
                return operator.success(valor)

            if self.method_name == "addIndex":
                chave = operator.registry(self.params.visit(operator))
                if operator.error: return operator
                valor = obj.addIndex(chave, self.value)
                return operator.success(valor)

            if self.method_name == "remove":
                if len(self.params) != 1:
                    return operator.fail(Error("Metodo 'remove' recebe exatamente 1 argumento"))
                value = operator.registry(self.params[0].visit(operator))
                if operator.error: return operator
                obj.removeItem(value)
                return operator.success(obj)
```

Foi adicionada a função `MethodListTuple`, que identifica as chamadas de funções e separa o nome da função chamada. Essas informações são enviadas para `NoMethodCall`, que é responsável por identificar a função que será chamada, assim como verificar se o objeto que a chamou tem suporte para essa função.

- **Novas funções para `TNumber`**

`TValue.py`

```
def equals(self, other):
    if isinstance(other, TNumber):
        return TNumber(self.value == other.value).setMemory(self.memory), None
    return super().pow(other)

def gt(self, other):
    if isinstance(other, TNumber):
        return TNumber(self.value > other.value).setMemory(self.memory), None
    return super().pow(other)

def lt(self, other):
    if isinstance(other, TNumber):
        return TNumber(self.value < other.value).setMemory(self.memory), None
    return super().pow(other)

def neq(self, other):
    if isinstance(other, TNumber):
        return TNumber(self.value != other.value).setMemory(self.memory), None
    return super().pow(other)

def lte(self, other):
    if isinstance(other, TNumber):
        return TNumber(self.value <= other.value).setMemory(self.memory), None
    return super().pow(other)

def gte(self, other):
    if isinstance(other, TNumber):
        return TNumber(self.value >= other.value).setMemory(self.memory), None
    return super().pow(other)
```

`SematicVisitor.py`

```
elif self.opTok.type == Consts.EQUALS:
    result, error = esq.equals(dir)
elif self.opTok.type == Consts.LT:
    result, error = esq.lt(dir)
elif self.opTok.type == Consts.GT:
    result, error = esq.gt(dir)
elif self.opTok.type == Consts.NEQ:
    result, error = esq.neq(dir)
elif self.opTok.type == Consts.LTE:
    result, error = esq.lte(dir)
elif self.opTok.type == Consts.GTE:
    result, error = esq.gte(dir)
```

Agora é possível realizar comparações entre valores numéricos, como `==`, `>=`, `<=`, entre outros, o que viabiliza a implementação de novas funcionalidades, como o tipo booleano, estruturas de condição e até mesmo estruturas de repetição. Com essa base sólida, podemos avançar para a criação de lógica de controle no compilador, permitindo a implementação de fluxos condicionais e loops que são essenciais em linguagens de programação.

4. Análise e Discussão

A implementação dos novos tipos de dados, como Listas e Tuplas, não apresentou grandes dificuldades, sendo realizadas de forma relativamente direta. No entanto, a criação de novas chamadas de métodos para cada tipo de dado foi um desafio maior, exigindo uma releitura detalhada do código para garantir que os métodos fossem corretamente integrados e funcionassem conforme esperado. A implementação das funções de comparação de valores numéricos, como `>`, `<`, `>=`, entre outras, também demandou um esforço significativo, pois envolveu garantir a compatibilidade desses operadores com os novos tipos de dados, algo que exigiu mais atenção aos detalhes do código já existente.

Adicionalmente, o projeto incluiu a intenção de implementar expressões booleanas, um passo importante para o avanço da linguagem. Apesar de entender os conceitos básicos da lógica booleana, como operadores **AND**, **OR** e **NOT**, a implementação completa dessa funcionalidade não foi possível. A lógica necessária para a execução correta dessas expressões envolve, entre outras coisas, a construção de um sistema que trate as variáveis booleanas de forma eficiente e permita realizar operações lógicas sobre elas. A dificuldade em implementar essa parte residiu principalmente na complexidade de adaptar o compilador para lidar com tais expressões de forma integrada.

5. Conclusão

Este trabalho proporcionou uma experiência enriquecedora, permitindo entender melhor como funciona o processo de desenvolvimento de um compilador e os desafios envolvidos em cada etapa. Embora as implementações de novos tipos de dados e funções de comparação tenham sido bem-sucedidas, a introdução de expressões booleanas revelou a complexidade adicional de se projetar um compilador. A experiência de lidar com os problemas práticos e as dificuldades que surgem durante o desenvolvimento de um compilador foi fundamental para compreender as dificuldades enfrentadas por projetistas e a importância de cada decisão tomada durante o processo de criação de uma linguagem de programação. Este trabalho também reforçou a necessidade de atenção aos detalhes e uma abordagem meticulosa para garantir a integração e a funcionalidade do código em diferentes módulos.